Interacting With Data Sources

DataFrameReader

- `DataFrameReader` is the core construct for reading data from a data source into a `DataFrame`. Recommended format:

```
DataFrameReader.format(args).option("key", "value")
               .schema(args).load()
```

- We can access a `DataFrameReader` through a `SparkSession` instance by using one either `SparkSession.read` or `SparkSession.readStream`.
- In general, we don't need a schema when reading a Parquet file as it is contained in its metadata. Parquet is the preferred data source because it is efficient, uses columnar storage, and employs a fast compression algorithm.

DataFrameReader Parameters

Method	Arguments	Description
`format()`	`"parquet", "csv", "txt", "json", "jdbc", "orc", "avro"` etc	Default is parquet or as specified in the config for `spark.sql.sources.default`.
`option()`	`("mode", {PERMISSIVE | FAILFAST | DROPMALFORMED )`	default is `PERMISSIVE`
	`("inferSchema", {true | false})`	`mode` and `inferSchema` is for JSON and CSV sources
	`("path", "path_file_data_source")`	For JSON and CSV formats
`schema()`	DDL String or StructType	
`load()`	`"path/to/data/source"`	optional if already specified above

DataFrameReader Parameters (cont.)

```
# Use Parquet
file = """flights/data/summary-data/parquet/2010-summary.parquet"""
df = spark.read.format("parquet").load(file)

# Use Parquet; you can omit format("parquet") if you wish as it's
    the default
df2 = spark.read.load(file)

# Use CSV
df3 = (spark.read.format("csv")
       .option("inferSchema", "true")
       .option("header", "true")
       .option("mode", "PERMISSIVE")
       .load("flights/data/summary-data/csv/*"))

# Use JSON
df4 = (spark.read.format("json")
       .load("flights/data/summary-data/json/*"))
```

DataFrameWriter

- `DataFrameWriter` is the core construct for writing data from a DataFrame to external source a DataFrame. Recommended formats:

```
DataFrameWriter.format(args)
                .option("key", "value")
                .bucketBy(args)
                .partitionBy(args)
                .save(path)
```

```
DataFrameWriter.format(args).option("key", "value")
               .sortBy(args).saveAsTable(table)
```

- We can access a `DataFrameWriter` directly through a `DataFrame` instance by using either `DataFrame.write` or `DataFrame.writeStream`.

DataFrameWriter Parameters

Method	Arguments
`format()`	`"parquet", "csv", "txt", "json",` `"jdbc", "orc", "avro" etc`

`option()`	`("mode", {append | overwrite`
	`| ignore | error or errorifexists} )`
	`("mode", {SaveMode.Overwrite`
	`| SaveMode.Append | SaveMode.Ignore}`
	`| SaveMode.ErrorIfExists})`
	`("inferSchema", {true | false})`
	`("path", "path_to_write_to")`

`bucketBy()`	#buckets and names of columns to bucket by
`save()`	`"path/to/data/source"`
`saveAsTable()`	`"table_name"`

Overview of Data Source Types

- Parquet
- JSON
- CSV
- Avro: Used by Apache Kafka for serialization. Offers direct mapping to JSON, speed and efficiency, and bindings for many programming languages.
- ORC (Optimized Row Columnar) file format provides a highly efficient way to store Hive data.
- Image data
- Arbitrary binary data: The DataFrame has the following attributes: path, modificationTime, length, and content. Writing isn't supported.

Parquet

- Parquet is a columnar storage format that is available to any project in the Hadoop ecosystem, regardless of the choice of data processing framework, data model, or programming language.
- Parquet is a popular choice for data storage and processing because it is efficient, uses columnar storage, and employs a fast compression algorithm.
- Parquet is the default data source format for Spark.

Interacting with External Data Sources

We will examine how Spark SQL interfaces with external components. Specifically, we will discuss how Spark SQL allows you to:

- Use user-defined functions for both Apache Hive and Apache Spark.
- Connect with external data sources such as JDBC and SQL databases, PostgreSQL, MySQL, Tableau, Azure Cosmos DB, and MS SQL Server.
- Work with simple and complex types, higher-order functions, and common relational operators.

Querying with Spark SQL Shell and Beeline

- Spark SQL provides a command line interface called the Spark SQL shell. It is similar to the Spark shell, but it is designed for running SQL queries. It can be started by running

```
spark-sql
```

- Now we can use it interactuvely to run SQL queries against data stored in Hive, Parquet, JSON, and other formats. Let's create a simple table and insert some data into it.

```
spark-sql> CREATE TABLE people (name STRING, age INT);
spark-sql> INSERT INTO people VALUES ('Alice', 30);
spark-sql> INSERT INTO people VALUES ('Michael', NULL);
spark-sql> INSERT INTO people VALUES ('Samantha', 39);
```

- Now we can query the table using SQL queries.

```
spark-sql> SHOW TABLES;
spark-sql> SELECT * FROM people;
spark-sql> SELECT name FROM people WHERE age IS NULL;
spark-sql> quit;
```

Querying with Spark SQL Shell and Beeline

- Spark SQL also provides a command line interface called Beeline. It is similar to the Spark SQL shell, but it is designed for running SQL queries against Hive.

```
start-all.sh --> if Spark isn't already started
start-thriftserver.sh
beeline
```

- Then connect to the Hive server using the following command:

```
beeline> !connect jdbc:hive2://localhost:10000
```

- Now we can use it interactively to run SQL queries against data stored in Hive.

```
beeline> SHOW TABLES;
beeline> SELECT * FROM people;
beeline> SELECT name FROM people WHERE age IS NULL;
beeline> !quit
```

External Data Sources

- Spark SQL can connect to external data sources using JDBC and ODBC. This allows you to query data stored in relational databases such as PostgreSQL, MySQL, and MS SQL Server.
- Spark SQL can also connect to external data sources such as Tableau and Azure Cosmos DB. This allows you to query data stored in these systems using SQL.
- Spark SQL can also connect to external data sources such as Cassandra, MongoDB, and Snowflake. This allows you to query data stored in these systems using SQL.